

Lecture 1 — Python Fundamentals Refresher (FinTech)

- Variables & Data Types
- Operators & Expressions
- Control Flow (if/elif/else)
- Loops & Iteration
- Functions (Basics)
- Mini FinTech practice throughout

Variables & Data Types

- Numbers (int/float) and strings
- Decimal for money (preview); formatted outputs

```
from decimal import Decimal

currency = Decimal("1234.50")    # exact money
rate_pct = 12.5                  # as float (safe for simple calcs)
years = 2

student_name = "Aiman"
print(f"{student_name} → Rs {currency} at {rate_pct}% for {years} years")
```

Input Cleaning (Strings → Numbers)

- Strip commas/spaces before converting to float/Decimal

```
def to_float_clean(text: str) -> float:  
    return float(text.replace(",", "").strip())
```

```
amount = to_float_clean(" 12,345.67 ")  
print(amount)  # 12345.67
```

Operators & Expressions

- Arithmetic, comparison, logical, membership
- Profit margin example

```
revenue = 250000
cost = 180000
profit_margin = (revenue - cost) / revenue
print(f"Profit Margin: {profit_margin:.2%}") # 28.00%
```

If / Elif / Else — Loan Eligibility

- Use thresholds to make decisions
- 650+ eligible, else not eligible

```
def loan_eligibility(credit_score: int) -> str:
    if credit_score >= 750:
        return "Prime borrower"
    elif credit_score >= 650:
        return "Eligible"
    else:
        return "Not eligible"

print(loan_eligibility(720))
print(loan_eligibility(610))
```

Guard Clauses & Tiered Fees

- Return early for invalid inputs
- Tier: > 10,000 → 2% fee, else flat Rs. 50

```
def net_withdrawal(amount: float) -> tuple[float, float]:  
    if amount <= 0:  
        return 0.0, 0.0 # guard clause  
    fee = 0.02 * amount if amount > 10_000 else 50.0  
    return amount - fee, fee  
  
print(net_withdrawal(5000))    # (4950.0, 50.0)  
print(net_withdrawal(20000))  # (19600.0, 400.0)
```

Loops — Simulate Price Path

- Apply daily % returns to a price series
- Use for-loop over list

```
price = 100.0
returns_pct = [0.5, -0.2, 0.3, 1.0] # percent moves

history = [price]
for r in returns_pct:
    price *= (1 + r/100)
    history.append(round(price, 2))

print("Trajectory:", history)
```

Loop Patterns — enumerate & early exit

- Track index and value together
- Break when condition is met

```
prices = [100, 102, 101, 98, 97, 103]
threshold = 99

for day, p in enumerate(prices, start=1):
    if p < threshold:
        print(f"Alert: fell below {threshold} on day {day} → {p}")
        break
```


Functions (Basics) — Simple Interest

- Reusable calculations
- Return multiple values if needed

```
def simple_interest(principal: float, rate_pct: float, years: float):  
    r = rate_pct / 100  
    si = principal * r * years  
    total = principal + si  
    return round(si, 2), round(total, 2)  
  
si, total = simple_interest(100000, 12, 2)  
print("SI:", si, "Total:", total)
```

Pure vs. Impure Functions (I/O)

- Pure: output depends only on inputs; easier to test
- Impure: uses input(), print(), or external state

```
# Pure
def add_fee(amount: float, fee: float) -> float:
    return amount + fee
```

```
# Impure (reads input)
# user_amount = float(input("Amount: "))
```

Mini Practice — Average Price

- Compute average closing price safely
- Handle empty list

```
def avg_price(prices: list[float]) -> float:
    if not prices:
        raise ValueError("No prices")
    return sum(prices) / len(prices)

print(round(avg_price([120, 118.5, 122.8, 121.0, 125.3]), 2))
```

Bug Hunt — UnboundLocalError

- Initialize before use; respect scopes

```
total = 0
def add_price(price):
    global total # or pass/return total explicitly
    total = total + price

add_price(100)
print(total)
```

Lecture 2 — Collections, Robust Code, Bridge to OOP

- Lists & Tuples
- Dictionaries & Sets
- Functions++ (defaults & multiple returns)
- Exceptions
- Comprehensions
- Bank Account (dict) → next: Class

Lists & Tuples — Create & Slice

- Indexing and slicing for series
- Tuple is immutable (safer for constants)

```
prices = [120, 122, 121, 125, 127]
first_three = prices[:3]
last = prices[-1]
pair: tuple[int, int] = (2024, 2025)
print(first_three, last, pair)
```

List Aggregations — Max, Min, Avg

- Write a small utility to compute stats

```
def stock_stats(prices):  
    if not prices:  
        raise ValueError("Empty series")  
    return max(prices), min(prices), sum(prices)/len(prices)  
  
mx, mn, avg = stock_stats([120, 122, 121, 125, 127])  
print(mx, mn, round(avg, 2))
```

Dictionaries — Account Balances

- Map account → balance
- Compute interest without mutation (functional)

```
accounts = {"A001": 5000.0, "A002": 10000.0, "A003": 7500.0}
```

```
def apply_interest(accts: dict[str, float], rate_pct: float) -> dict[str, float]:  
    factor = 1 + rate_pct/100  
    return {acc: round(bal * factor, 2) for acc, bal in accts.items()}
```

```
print(apply_interest(accounts, 5))
```


Sets — Unique Symbols

- Deduplicate portfolio tickers

```
portfolio = ["HBL", "UBL", "HBL", "BAHL", "UBL"]  
unique = set(portfolio)  
print(unique)
```

Functions with Defaults & Multiple Returns

- Design clear APIs
- Loan summary: total payable + annual installment

```
def loan_summary(principal: float, rate_pct: float = 10.0, years: int = 5):  
    r = rate_pct/100  
    total = principal * (1 + r * years) # simple interest approx  
    annual_installment = total / years  
    return round(total, 2), round(annual_installment, 2)  
  
print(loan_summary(100000))
```

Exceptions — Defensive Division

- Guard against bad input and zero division

```
try:
    d = float(input("Divisor: "))
    result = 1000 / d
except ValueError:
    print("Please enter a numeric divisor.")
except ZeroDivisionError:
    print("Divisor cannot be zero.")
else:
    print(f"Result = {result:.2f}")
```

Comprehensions — Portfolio KPIs

- Concise aggregations over lists/dicts
- Compute market value & average price per symbol

```
portfolio = [  
    {"symbol": "HBL", "qty": 10, "price": 95.5},  
    {"symbol": "UBL", "qty": 5, "price": 118.2},  
    {"symbol": "HBL", "qty": 2, "price": 96.1},  
]  
  
symbols = {p["symbol"] for p in portfolio}  
market_value = sum(p["qty"] * p["price"] for p in portfolio)  
print(symbols, round(market_value, 2))
```

Dict Aggregation — Avg Price per Symbol

- Use defaultdict to accumulate then average

```
from collections import defaultdict

portfolio = [
    {"symbol": "HBL", "qty": 10, "price": 95.5},
    {"symbol": "UBL", "qty": 5, "price": 118.2},
    {"symbol": "HBL", "qty": 2, "price": 96.1},
]

sum_price, count = defaultdict(float), defaultdict(int)
for p in portfolio:
    s = p["symbol"]
    sum_price[s] += p["price"]
    count[s] += 1

avg_price = {s: round(sum_price[s]/count[s], 2) for s in sum_price}
print(avg_price)
```

Bank Account (Dict + Functions) — Bridge to OOP

- Encapsulate operations on one account
- Next lecture: refactor to a BankAccount class

```
account = {"name": "Ali", "balance": 10_000.0}

def deposit(acc: dict, amount: float) -> None:
    if amount <= 0:
        raise ValueError("Deposit must be positive")
    acc["balance"] += amount

def withdraw(acc: dict, amount: float, overdraft_limit: float = 0.0) -> bool:
    if amount <= 0:
        raise ValueError("Withdrawal must be positive")
    if acc["balance"] - amount < -overdraft_limit:
        return False
    acc["balance"] -= amount
    return True

deposit(account, 2500)
ok = withdraw(account, 13000, overdraft_limit=1000)
print("OK?" , ok, "Balance:", account["balance"])
```

Preview — Class Skeleton (Next Time)

- Data + behavior bind together
- Constructor, methods, encapsulation

```
class BankAccount:
    def __init__(self, name: str, balance: float = 0.0):
        self.name = name
        self.balance = balance

    def deposit(self, amount: float) -> None:
        ... # implement next lecture

    def withdraw(self, amount: float) -> bool:
        ...

# to come: statements, interest, validation
```